Dr. RVR NRI INSTITUTE OF TECHNOLOGY

Department of Computer Science & Engineering | Dussehra ABL Assignment - Oct 2026

Student Name	Thanush Lanke	**Registration No**	26000176
Course Name	Python Programming (ABL)	**Branch & Section**	CSE Core - 1st Year
GitHub Repository	python-activity-based-learning	**Evaluation Marks**	10 Marks

10 SELECTED CASE STUDIES - CONSOLIDATED REPORT

Task 1: Progressive Electricity Billing (Task 2)

1. Problem Statement: Calculate household electricity bills using progressive consumption slabs (0-100 units @ Rs. 2.0, 101-200 @ Rs.3.5, >200 @ Rs.5.0) with a fixed meter charge of Rs.50.

2. Objectives: Apply tiered multi-slab conditional branching to model realistic utility billing logic.

3. Assumptions: Input units are non-negative integers. Meter charges are uniformly applicable.

4. Algorithm & Pseudocode:

START -> Read units -> IF units <= 100 THEN bill = units*2 + 50 ELSIF units <= 200 THEN bill = 200 + (units-100)*3.5 + 50 ELSE bill = 200 + 350 + (units-200)*5 + 50 -> Output bill -> END

5. Python Source Code:

```
# Task 2: Progressive Electricity Billing
units = int(input("Enter units consumed: "))
fixed_charge = 50

if units <= 100:
    bill = (units * 2.0) + fixed_charge
elif units <= 200:
    bill = (100 * 2.0) + ((units - 100) * 3.5) + fixed_charge
else:
    bill = (100 * 2.0) + (100 * 3.5) + ((units - 200) * 5.0) + fixed_charge

print("Total Electricity Bill: Rs.", bill)
```

6. Test Cases & Output Screenshots:

```
TERMINAL OUTPUT SCREENSHOT - CASE 1: 250 UNITS
Enter units consumed: 250
Total Electricity Bill: Rs. 800.0
```

7. Conclusion: The program successfully computes multi-tier progressive billing with exact precision.

Task 2: ATM Transaction Simulation (Task 13)

1. Problem Statement: Simulate interactive ATM operations (deposit/withdrawal) and update account balance while ensuring funds availability.

2. Objectives: Implement user input processing and balance validation rules.

3. Assumptions: Starting balance is initialized to Rs. 5000.0.

4. Algorithm & Pseudocode:

START -> Display balance -> Read operation choice ('d'/'w') and amount -> IF choice == 'd' THEN add amount ELSIF choice == 'w' AND amount <= balance THEN subtract amount ELSE print error -> Display updated balance -> END

5. Python Source Code:

```
# Task 13: ATM Transaction Simulation
balance = 5000.0
print("Current Balance:", balance)

choice = input("Enter 'd' to deposit or 'w' to withdraw: ")
amount = float(input("Enter amount: "))

if choice == 'd':
    balance += amount
    print("Updated Balance:", balance)
elif choice == 'w' and amount <= balance:
    balance -= amount
    print("Updated Balance:", balance)
else:
    print("Invalid transaction or insufficient balance.")
```

6. Test Cases & Output Screenshots:

TERMINAL OUTPUT SCREENSHOT - WITHDRAWAL CASE

```
Current Balance: 5000.0
Enter 'd' to deposit or 'w' to withdraw: w
Enter amount: 1200
Updated Balance: 3800.0
```

7. Conclusion: Validated transaction execution and balance tracking logic.

Task 3: Attendance Recovery Computation (Task 12)

1. Problem Statement: Compute current attendance percentage and determine academic exam eligibility against a 75% threshold.

2. Objectives: Practice mathematical percentage calculation and threshold checking.

3. Assumptions: Total classes held > 0. Attended classes <= total classes.

4. Algorithm & Pseudocode:

START -> Read attended, total -> percentage = (attended/total)*100 -> IF percentage >= 75 THEN print eligible ELSE print shortage -> END

5. Python Source Code:

```
# Task 12: Attendance Recovery Computation
attended = int(input("Enter classes attended: "))
total = int(input("Enter total classes held: "))

percentage = (attended / total) * 100
print(f"Current Attendance: {percentage:.2f}%")

if percentage >= 75:
    print("Eligible for exams.")
else:
    print("Attendance shortage! Needs improvement.")
```

6. Test Cases & Output Screenshots:

```
TERMINAL OUTPUT SCREENSHOT
Enter classes attended: 45
Enter total classes held: 50
Current Attendance: 90.00%
Eligible for exams.
```

7. Conclusion: Accurately checks percentage and flags shortages.

Task 4: Password Policy Validation Engine (Task 29)

1. Problem Statement: Evaluate security compliance of password strings against a minimum length standard of 8 characters.

2. Objectives: Execute string length evaluation using `len()` function.

3. Assumptions: User inputs a single string.

4. Algorithm & Pseudocode:

START -> Read password string -> IF len(password) >= 8 THEN print validation success ELSE print too short -> END

5. Python Source Code:

```
# Task 29: Password Policy Validation Engine
password = input("Enter your password: ")

if len(password) >= 8:
    print("Password validation successful.")
else:
    print("Password is too short. Must be at least 8 characters.")
```

6. Test Cases & Output Screenshots:

```
TERMINAL OUTPUT SCREENSHOT
Enter your password: securePass123
Password validation successful.
```

7. Conclusion: Basic password policy length criteria enforced.

Task 5: Multi-Product Shopping Budget Constraint (Task 19)

1. Problem Statement: Calculate combined cost of multiple items and compare total against maximum budget.

2. Objectives: Multi-variable price summation and inequality checks.

3. Assumptions: Budget and item prices are positive float values.

4. Algorithm & Pseudocode:

START -> Read budget, item1_price, item2_price -> total = item1 + item2 -> IF total <= budget THEN print within budget ELSE print budget exceeded -> END

5. Python Source Code:

```
# Task 19: Multi-Product Shopping Budget Constraint
budget = float(input("Enter total budget: "))
item1_price = float(input("Enter price of item 1: "))
item2_price = float(input("Enter price of item 2: "))

total_cost = item1_price + item2_price
print("Total Cost:", total_cost)

if total_cost <= budget:
    print("Purchase is within your budget!")
else:
    print("Budget exceeded! Reduce items.")
```

6. Test Cases & Output Screenshots:

```
TERMINAL OUTPUT SCREENSHOT
Enter total budget: 1000
Enter price of item 1: 350
Enter price of item 2: 450
Total Cost: 800.0
Purchase is within your budget!
```

7. Conclusion: Budget constraints verified effectively.

Task 6: Employee Payroll and Deduction Analysis (Task 4)

1. Problem Statement: Compute gross salary (Basic + 20% HRA + 10% DA), calculate progressive tax deductions (5%, 10%, 15%), and output net salary.

2. Objectives: Master multi-step payroll computations and progressive tax brackets.

3. Assumptions: Basic pay is provided as positive float.

4. Algorithm & Pseudocode:

START -> Read basic_pay -> hra = 0.2*basic, da = 0.1*basic -> gross = basic + hra + da -> IF gross <= 30000 tax=5% ELSIF gross <= 60000 tax=10% ELSE tax=15% -> net = gross - tax -> Display gross, tax, net -> END

5. Python Source Code:

```
# Task 4: Employee Payroll and Deduction Analysis
basic_pay = float(input("Enter basic pay: "))

hra = basic_pay * 0.20
da = basic_pay * 0.10
gross_salary = basic_pay + hra + da

if gross_salary <= 30000:
    tax = gross_salary * 0.05
elif gross_salary <= 60000:
    tax = gross_salary * 0.10
else:
    tax = gross_salary * 0.15

net_salary = gross_salary - tax
print("Gross Salary:", gross_salary)
print("Tax Deduction:", tax)
print("Net Salary:", net_salary)
```

6. Test Cases & Output Screenshots:

```
TERMINAL OUTPUT SCREENSHOT
Enter basic pay: 40000
Gross Salary: 52000.0
Tax Deduction: 5200.0
Net Salary: 46800.0
```

7. Conclusion: Automated salary slip calculations complete.

Task 7: Student Marks Matrix Analysis (Task 42)

1. Problem Statement: Analyze a 2D matrix of student subject marks to calculate row-wise totals and averages.

2. Objectives: Practice nested list iteration and multi-dimensional data processing.

3. Assumptions: Marks are stored in a pre-defined 3x3 nested list.

4. Algorithm & Pseudocode:

START -> Initialize marks_matrix -> FOR i in range(len(matrix)): total = sum(row), avg = total/len(row) -> Output student totals -> END

5. Python Source Code:

```
# Task 42: Student Marks Matrix Analysis
marks_matrix = [
    [75, 80, 85],
    [60, 65, 70],
    [90, 85, 95]
]

for i in range(len(marks_matrix)):
    total = sum(marks_matrix[i])
    avg = total / len(marks_matrix[i])
    print(f"Student {i+1} -> Total Marks: {total}, Average: {avg:.2f}")
```

6. Test Cases & Output Screenshots:

```
TERMINAL OUTPUT SCREENSHOT
Student 1 -> Total Marks: 240, Average: 80.00
Student 2 -> Total Marks: 195, Average: 65.00
Student 3 -> Total Marks: 270, Average: 90.00
```

7. Conclusion: 2D matrix iteration processed successfully.

Task 8: Smart Canteen Order and Stock Management (Task 57)

1. Problem Statement: Manage canteen inventory using dictionaries, process customer orders, calculate bills, and update available stock.

2. Objectives: Implement dictionary operations, key lookup, and inventory tracking.

3. Assumptions: Items are pre-stocked in a dictionary mapping.

4. Algorithm & Pseudocode:

START -> Read item and quantity -> IF item in stock AND qty <= stock[item] THEN deduct stock, print bill ELSE print error -> END

5. Python Source Code:

```
# Task 57: Smart Canteen Order and Stock Management
stock = {"Burger": 10, "Pizza": 5, "Tea": 20}
price = {"Burger": 50, "Pizza": 120, "Tea": 15}

item = input("Enter item to buy (Burger/Pizza/Tea): ")
qty = int(input("Enter quantity: "))

if item in stock and stock[item] >= qty:
    stock[item] -= qty
    total_bill = price[item] * qty
    print("Order successful! Total Bill: Rs.", total_bill)
    print("Remaining stock for", item, ":", stock[item])
else:
    print("Sorry, item out of stock or invalid name.")
```

6. Test Cases & Output Screenshots:

```
TERMINAL OUTPUT SCREENSHOT
Enter item to buy (Burger/Pizza/Tea): Burger
Enter quantity: 2
Order successful! Total Bill: Rs. 100
Remaining stock for Burger : 8
```

7. Conclusion: Inventory dictionary update validated.

Task 9: Library Borrowing and Fine Reconciliation (Task 58)

1. Problem Statement: Calculate overdue fines for late library book returns using tiered penalty slabs (1-5 days @ Rs. 2, 6-10 @ Rs.5, >10 @ Rs.10).

2. Objectives: Execute complex slab-based mathematical accumulation.

3. Assumptions: Overdue days are positive integer inputs.

4. Algorithm & Pseudocode:

START -> Read days_late -> IF days <= 5 fine=days*2 ELSIF days <= 10 fine=10+(days-5)*5 ELSE fine=35+(days-10)*10 -> Display fine -> END

5. Python Source Code:

```
# Task 58: Library Borrowing and Fine Reconciliation
days_late = int(input("Enter number of days book is overdue: "))

if days_late <= 5:
    fine = days_late * 2
elif days_late <= 10:
    fine = (5 * 2) + ((days_late - 5) * 5)
else:
    fine = (5 * 2) + (5 * 5) + ((days_late - 10) * 10)

print("Total Library Fine: Rs.", fine)
```

6. Test Cases & Output Screenshots:

```
TERMINAL OUTPUT SCREENSHOT
Enter number of days book is overdue: 12
Total Library Fine: Rs. 55
```

7. Conclusion: Tiered penalty logic computes exact fines.

Task 10: Check Even or Odd Number (Task 10)

1. Problem Statement: Determine whether a given integer is even or odd using the modulus operator (`%`).

2. Objectives: Apply fundamental arithmetic parity testing.

3. Assumptions: Input is a valid integer.

4. Algorithm & Pseudocode:

START -> Read num -> IF num % 2 == 0 THEN print Even ELSE print Odd -> END

5. Python Source Code:

```
# Task 10: Check Even or Odd Number
num = int(input("Enter a number: "))

if num % 2 == 0:
    print("The number is Even")
else:
    print("The number is Odd")
```

6. Test Cases & Output Screenshots:

```
TERMINAL OUTPUT SCREENSHOT
Enter a number: 18
The number is Even
```

7. Conclusion: Parity check verified.